SECURITY ASSESSMENT

Accretion

Provided by Accretion Labs Pte Ltd. for Solana Foundation
April 03, 2026
A26SFR2

Solana Foundation

AUDITORS

Role	Name
Lead Auditor	Niklas Brymko (niklas@accretion.xyz)
Lead Auditor	Robert Reith (robert@accretion.xyz)

CLIENT

Solana Foundation (https://solana.org) engaged Accretion to conduct a security assessment of a framework-agnostic Solana transaction signing library with multiple backend implementations including enterprise vault integration and managed wallet services.

ENGAGEMENT TIMELINE

16 Feb — **Project Kickoff**
Initial planning and scope definition

16 Feb — **Assessment Begins**
Security review and testing phase

01 Mar — **Review Fixes**
Security recommendations are given and implemented

01 Apr — **Project Completion**
Report delivery and on-chain confirmation

AUDITED CODE

#	Program ID	Repository
1	N/A	https://github.com/solana-foundation/solana-keychain

ASSESSMENT

The security assessment of Solana Foundation's Solana Keychain revealed 21 security issues across critical infrastructure components. The findings included 1 high-severity issue, 4 medium-severity issues, 13 low-severity issues, and 3 informational items. Issues spanned authentication handling, cryptographic key management, API security, and error handling practices. All identified vulnerabilities have been successfully remediated by the development team.

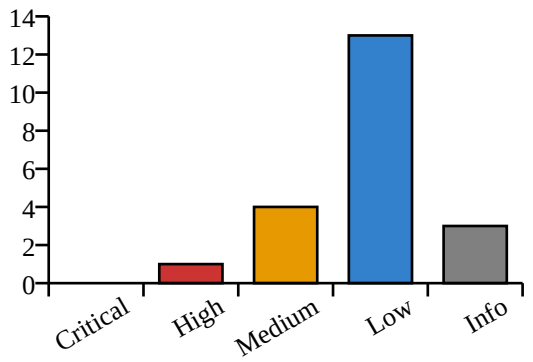
CODE ASSESSMENT

The codebase demonstrates solid architectural design with comprehensive remote signer integrations. However, several security practices needed improvement including HTTPS enforcement, proper error handling, and secure key material management across both Rust and TypeScript implementations.

KEY FINDINGS

One high-severity issue was identified: CDP Signer completely replaces caller transactions with remote API responses, enabling potential transaction manipulation attacks. This architectural flaw could allow malicious API responses to substitute intended transactions.

SEVERITY DISTRIBUTION



© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Solana Keychain Rust Library

Link: https://github.com/solana-foundation/solana-keychain
Commit: 4f67d1288efe62623e55c79f0cccce6c2c230784
Program ID: N/A
Audit Result:
- **Audited Commit:** 475beb7c372e805e081ebcd85d84923460267da6
- **Comment:** Software library

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-H1	CDP Signer Trusts and Replaces Entire Transaction from Remote API	HIGH	FIXED
ACC-M1	Vault Signature Version Prefix Only Strips `vault:v1:`	MEDIUM	FIXED
ACC-M2	`Display` Trait on `SignerError` Does Not Redact (Bypasses `Debug` Redaction)	MEDIUM	FIXED
ACC-M3	No HTTPS Enforcement on HTTP Clients (All Remote Signers)	MEDIUM	FIXED
ACC-M4	Panic in Production Code on Malformed API Response (Privy)	MEDIUM	FIXED
ACC-L1	Public API Functions Lack `@throws` Documentation [TS]	LOW	FIXED
ACC-L2	Error Context Includes Remote API Response Text [TS]	LOW	FIXED
ACC-L3	Fireblocks JWT Expiry of 30s Is Fragile with Clock Skew [TS]	LOW	FIXED
ACC-L4	Malformed API Responses Throw Raw `TypeError` Instead of `SignerError` [TS]	LOW	FIXED
ACC-L5	Vault Signature Version Prefix Only Strips `vault:v1:` [TS]	LOW	FIXED
ACC-L6	No HTTPS Enforcement on HTTP Clients (5 of 9 Signers) [TS]	LOW	FIXED
ACC-L7	Privy `is_available()` Does Not Check Remote API	LOW	FIXED
ACC-L8	Fireblocks JWT Expiry of 30s Is Fragile with Clock Skew	LOW	FIXED
ACC-L9	AWS KMS `is_available()` Does Not Check Key Enabled/Usage State	LOW	FIXED
ACC-L10	Intermediate Key Material Not Zeroed (Memory Signer)	LOW	FIXED
ACC-L11	Use-Before-Init Silently Uses Default Zero Public Key (Privy, Fireblocks)	LOW	FIXED
ACC-L12	Arbitrary File Read via `from_private_key_string` (Memory Signer)	LOW	FIXED
ACC-L13	`sign_transaction` and `sign_partial_transaction` are identical with no type-level distinction	LOW	FIXED
ACC-I1	Turnkey and Dfns Auth Use Node.js-Specific Crypto [TS]	INFO	FIXED
ACC-I2	Fireblocks RSA PEM Re-parsed on Every API Call	INFO	FIXED
ACC-I3	No HTTP Request Timeouts Configured (All Remote Signers)	INFO	FIXED

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

DETAILED ISSUES

ACC-H1 CDP Signer Trusts and Replaces Entire Transaction from Remote API

HIGH FIXED

Description

CDP's `sign_and_serialize()` deserializes the entire signed transaction from the CDP API response and overwrites the caller's transaction with `*transaction = signed_tx`. Unlike all other signers, which only inject a signature into the caller's existing transaction, CDP replaces the whole thing.

A compromised or malicious CDP API can return a transaction with different recipients, amounts, or instructions. The caller's original transaction is silently discarded.

Location

https://github.com/solana-foundation/solana-keychain/blob/d3ffc364c70429b9418568f72846a37999da7f4a/rust/src/cdp/mod.rs#L336-L378

Relevant Code

```
/// cdp/mod.rs L336-L378
let signed_tx: Transaction = bincode::deserialize(&signed_bytes).map_err(|_e| { ... })?;
let pos = TransactionUtil::get_signing_keypair_position(&signed_tx, &self.public_key)?;
let signature = *signed_tx.signatures.get(pos).ok_or_else(|| { ... })?;
*transaction = signed_tx;
```

Mitigation Suggestion

Extract only the signature from the CDP response and inject it into the caller's original transaction using `TransactionUtil::add_signature_to_transaction`, matching the pattern used by all other signers.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-04590420adf8264a667b569622d360d1a75b3f0386d5ad33a47cd14ea07df896R360.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M1 Vault Signature Version Prefix Only Strips `vault:v1:`

MEDIUM FIXED

Description

Key rotation in Vault produces `vault:v2:`, `vault:v3:`, etc. Only `v1` is handled. Rotated keys will cause silent failures with an uninformative "Failed to decode signature" error.

See: https://github.com/sigstore/cosign/issues/1351 https://developer.hashicorp.com/vault/api-docs/secret/transit https://developer.hashicorp.com/vault/docs/secrets/transit

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/vault/mod.rs#L114-L116

Relevant Code

```
/// mod.rs L114-L116
        let signature_b64 = signature_b64
            .strip_prefix("vault:v1:")
            .unwrap_or(signature_b64);
```

Mitigation Suggestion

Use a more general prefix-stripping approach, e.g., strip anything matching `vault:v\d+:`.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-1f2e9c5e2777c9e9464ca4af7fd3af7f39f9248343441db1059e1740cecc6a0aR43.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M2 `Display` Trait on `SignerError` Does Not Redact (Bypasses `Debug` Redaction)

MEDIUM FIXED

Description

We found that the custom `Debug` implementation carefully redacts all error payloads. However, the `thiserror`-generated `Display` implementation includes inner strings verbatim (e.g., `"Invalid private key format: {0}"`). Most Rust logging and error-handling patterns use `Display` (`{}` format, `.to_string()`, `anyhow`, `?` operator), making the redacted `Debug` implementation largely ineffective.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/error.rs#L7-L48
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/error.rs#L75-L98

Relevant Code

```rust
/// error.rs L7-L48 thiserror-generated Display includes inner strings verbatim
#[derive(Error)]
pub enum SignerError {
    #[error("Invalid private key format: {0}")]
    InvalidPrivateKey(String),
    // ...
}

/// error.rs L75-L98 -- custom Debug redacts everything
impl fmt::Debug for SignerError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            SignerError::InvalidPrivateKey(_) => {
                write!(f, "SignerError::InvalidPrivateKey([REDACTED])")
            }
            // ...
        }
    }
}
```

Mitigation Suggestion

Implement custom `Display` with redaction, or ensure inner strings never contain sensitive material.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-9b20a815681b9698e5b684bbd5ffc47ca944a7a80835e4572ec857c9a8e77fb8.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M3 No HTTPS Enforcement on HTTP Clients (All Remote Signers)

MEDIUM FIXED

Description

No signer validates that its `base_url` / `addr` uses HTTPS. A misconfigured HTTP URL sends authentication credentials in cleartext.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/vault/mod.rs#L44
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/privy/mod.rs#L46
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/turnkey/mod.rs#L60
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/fireblocks/mod.rs#L76-L78

Relevant Code

```
/// All remote signers use reqwest::Client::new() or cloud SDK defaults without enforcing TLS
```

Mitigation Suggestion

Validate URL scheme in constructors or use `reqwest::ClientBuilder::https_only(true)`. Given the Mock in the test will continue to rely on the old behaviour, it is recommended to cfg guard this to not catch this during `cargo test` like follows:

```
let builder = reqwest::Client::builder();

#[cfg(not(test))]
let builder = builder.https_only(true);

builder.build()
```

If the library user indeed wants this behaviour, you can add a configuration to disable this enforcement as an intentional security opt-out.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-8939ae905a0cf89e0d915460466b52d48b2937140410af4a7c2464b68e1fd36d.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M4 Panic in Production Code on Malformed API Response (Privy)

MEDIUM FIXED

Description

The `.expect()` call will panic and crash the host application if the Privy API returns a valid JSON response with a `signature` field that is not a valid Base64. This is the only `expect()` on untrusted external input in production code.

As a library, this code should never cause panics, and no other adapter is causing panics on failure either.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/privy/mod.rs#L144-L146

Relevant Code

```
let decoded_response = STANDARD
    .decode(&sign_response.data.signature)
    .expect("Failed to decode response");
```

Mitigation Suggestion

Replace with `map_err(|e| SignerError::SerializationError(...))?`.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-cbe50073e333b387b274cd03aac1049274f3588d76f1448d5b37c337ff2859c6R175.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L1 Public API Functions Lack `@throws` Documentation [TS]

LOW FIXED

Description

Only 5 `@throws` tags exist across ~5,800 lines of source. None of the public factory functions (`createPrivySigner`, `createVaultSigner`, `createTurnkeySigner`, etc.) documents what they throw. The `SolanaSigner` interface methods (`signMessages`, `signTransactions`, `isAvailable`) have no `@throws` either.

The core README shows `SignerError` codes and an `instanceof SignerError` check, but doesn't say which functions throw which codes. Callers have no way to know what to catch. A caller writing `catch (e) { if (e instanceof SignerError) ... }` will silently swallow or miss errors depending on which signer they're using and what the API returns.

TypeScript has no checked exceptions, so `@throws` JSDoc is the only mechanism to communicate this. Without it, error handling is guesswork.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/privy/src/privy-signer.ts#L31
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/vault/src/vault-signer.ts#L22
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/turnkey/src/turnkey-signer.ts#L22 https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/fireblocks/src/fireblocks-signer.ts#L29 https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/aws-kms/src/aws-kms-signer.ts#L16 https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/gcp-kms/src/gcp-kms-signer.ts#L16
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/dfns-signer.ts#L27
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/cdp/src/cdp-signer.ts#L24
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/para/src/para-signer.ts#L16
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/core/src/types.ts#L28
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/core/src/types.ts#L37
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/core/src/types.ts#L46

Relevant Code

```
// privy-signer.ts L31
export async function createPrivySigner<TAddress extends string = string>(
    config: PrivySignerConfig,
): Promise<SolanaSigner<TAddress>> { ... }

// core/types.ts L37
signMessages(messages: readonly SignableMessage[]): Promise<readonly SignatureDictionary[]>;

// Compare: only 5 @throws in the entire SDK, all in core/utils.ts and one in fireblocks
```

Mitigation Suggestion

Add `@throws {SignerError}` to every public function with the specific error codes it can produce.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/commit/8ee0909.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L2 Error Context Includes Remote API Response Text [TS]

LOW FIXED

Description

All signers include the remote API's error response text in the `SignerError` context object. If a malicious or misconfigured server returns error messages containing reflected input (e.g., echoing back a token or key material), those strings propagate into the error and may end up in logs.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/privy/src/privy-signer.ts#L168
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/privy/src/privy-signer.ts#L231
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/privy/src/privy-signer.ts#L371
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/turnkey/src/turnkey-signer.ts#L200 https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/turnkey/src/turnkey-signer.ts#L306 https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/fireblocks/src/fireblocks-signer.ts#L201 https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/fireblocks/src/fireblocks-signer.ts#L265
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/dfns-signer.ts#L315
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/dfns-signer.ts#L377
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/auth.ts#L50
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/auth.ts#L120
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/cdp/src/cdp-signer.ts#L376
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/cdp/src/cdp-signer.ts#L435
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/vault/src/vault-signer.ts#L172
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/para/src/para-signer.ts#L337

Relevant Code

```
throwSignerError(SignerErrorCode.REMOTE_API_ERROR, {
    url,
    status: response.status,
    response: errorText,  // <-- remote server controls this string
});
```

Mitigation Suggestion

Truncate or sanitize error response text before including it in error context.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/commit/8ee0909.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L3 Fireblocks JWT Expiry of 30s Is Fragile with Clock Skew [TS]

LOW FIXED

Description

Same issue as Rust. The Fireblocks JWT has a 30-second lifetime with no `nbf` claim. Clock skew > 30s between client and Fireblocks causes every request to fail with an opaque auth error.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/fireblocks/src/jwt.ts#L37

Relevant Code

```
exp: Math.floor(Date.now() / 1000) + 30,
```

Mitigation Suggestion

Extend lifetime or add an `nbf` claim with leeway.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/commit/8ee0909.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L4 Malformed API Responses Throw Raw `TypeError` Instead of `SignerError` [TS]

LOW FIXED

Description

Every signer wraps `response.json()` in a try/catch to handle malformed JSON, then casts the result with `as SomeType`. But the property access on the cast result happens outside the try/catch. If an API returns valid JSON with an unexpected shape (e.g., `{}` instead of `{ activity: { result: ... } }`), the unguarded dot access throws a raw `TypeError` that escapes into caller code.

Callers doing `catch (e) { if (e instanceof SignerError) ... }` will miss these entirely. The error isn't structured, has no error code, and the stack trace points at an internal line with no useful context. A misbehaving or compromised API server can trigger this at will.

Vault, Para, Privy, and CDP are safe because they either use optional chaining (`?.`) or null-check before access.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/turnkey/src/turnkey-signer.ts#L215 https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/fireblocks/src/fireblocks-signer.ts#L216
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/dfns-signer.ts#L330
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/auth.ts#L66

Relevant Code

```
// turnkey-signer.ts L205-215
try {
    activityResponse = (await response.json()) as ActivityResponse;
} catch (error) {
    throwSignerError(SignerErrorCode.PARSING_ERROR, { ... });
}
const signResult = activityResponse.activity.result?.signRawPayloadResult;

// fireblocks-signer.ts L206-216
try {
    addressesResponse = (await response.json()) as VaultAddressesResponse;
} catch (error) {
    throwSignerError(SignerErrorCode.PARSING_ERROR, { ... });
}
const firstAddress = addressesResponse.addresses[0]?.address;

// dfns-signer.ts L320-330
try {
    sigResponse = (await response.json()) as GenerateSignatureResponse;
} catch (error) {
    throwSignerError(SignerErrorCode.PARSING_ERROR, { ... });
}
if (sigResponse.status === 'Failed') { ... }
```

Mitigation Suggestion

Wrap the entire parse-and-access block in the try/catch, not just `response.json()`. Alternatively, validate the response shape before accessing nested properties:

```
try {
    const raw = await response.json();
    if (!raw?.activity?.result) {
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
        throwSignerError(SignerErrorCode.PARSING_ERROR, {
            message: 'Unexpected response shape from Turnkey API',
        });
    }
    activityResponse = raw as ActivityResponse;
} catch (error) {
    if (error instanceof SignerError) throw error;
    throwSignerError(SignerErrorCode.PARSING_ERROR, {
        cause: error,
        message: 'Failed to parse Turnkey response',
    });
}
```

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/commit/8ee0909 and https://github.com/solana-foundation/solana-keychain/commit/fc44053.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L5 Vault Signature Version Prefix Only Strips `vault:v1:` [TS]

LOW FIXED

Description

Same issue as in the Rust component. Key rotation in Vault produces `vault:v2:`, `vault:v3:`, etc. The TypeScript implementation only checks for `vault:v1:`. After a key rotation, the prefix won't be stripped, the remaining string isn't a valid base64, and decoding fails with a confusing error.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/vault/src/vault-signer.ts#L124

Relevant Code

```
// vault-signer.ts L124
const base64Signature = vaultSignature.startsWith('vault:v1:')
    ? vaultSignature.slice('vault:v1:'.length)
    : vaultSignature;
```

Mitigation Suggestion

Use a regex to strip any version prefix: `vaultSignature.replace(/^vault:v\d+:/, '')`.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/commit/8ee0909.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L6 No HTTPS Enforcement on HTTP Clients (5 of 9 Signers) [TS]

LOW FIXED

Description

Privy, Vault, Turnkey, Fireblocks, and Dfns all accept an `apiBaseUrl` config field as a plain string with no scheme validation. Only Para enforces HTTPS. AWS KMS and GCP KMS use official cloud SDKs that handle TLS internally.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/privy/src/privy-signer.ts https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/vault/src/vault-signer.ts https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/turnkey/src/turnkey-signer.ts https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/fireblocks/src/fireblocks-signer.ts https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/dfns-signer.ts

Relevant Code

```
// Para (para-signer.ts) DOES this check
if (parsedUrl.protocol !== 'https:') {
    throwSignerError(SignerErrorCode.CONFIG_ERROR, {
        message: 'apiBaseUrl must use HTTPS',
    });
}
```

Mitigation Suggestion

Add the same URL scheme check Para uses to every signer that takes a base URL config. Parse with `new URL()`, reject anything that isn't `https:`.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/commit/fc44053#diff-1f000257b37bd940443b36abbb68dd120f6bb43449b7ab4c053b1a663a5d609b.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L7 Privy `is_available()` Does Not Check Remote API

LOW FIXED

Description

All remote signers implement `is_available()` by making an API call to verify the backend is reachable and credentials are valid. Privy is the sole exception. It only checks whether `init()` has been called by comparing the cached public key against `Pubkey::default()`.

This means `is_available()` returns `true` even when the Privy API is down, credentials have been revoked, or the wallet has been deleted.

Location

https://github.com/solana-foundation/solana-keychain/blob/b302537/rust/src/privy/mod.rs#L199-L202

Relevant Code

```
async fn is_available(&self) -> bool {
    self.public_key != Pubkey::default()
}
```

Mitigation Suggestion

Make `is_available()` call the Privy wallet endpoint (e.g., `GET /wallets/{wallet_id}`) and verify the response succeeds, matching the pattern used by all other remote signers.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-cbe50073e333b387b274cd03aac1049274f3588d76f1448d5b37c337ff2859c6R250.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L8 Fireblocks JWT Expiry of 30s Is Fragile with Clock Skew

LOW FIXED

Description

30-second JWT lifetime with no `nbf` claim. Clock skew >30s between client and server causes all requests to fail.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/fireblocks/jwt.rs#L45

Relevant Code

```
/// jwt.rs L41-L48
    let claims = FireblocksClaims {
        uri: uri.to_string(),
        nonce: Uuid::new_v4().to_string(),
        iat: now,
        exp: now + 30,
        sub: api_key.to_string(),
        body_hash,
    };
```

Mitigation Suggestion

Extend lifetime or add an `nbf` claim with leeway.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-4a67943159e45b3b9349e42ef2f8f2f5b1a249d05bcbdd18e48c0d7354ebc69aR49.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L9 AWS KMS `is_available()` Does Not Check Key Enabled/Usage State

LOW FIXED

Description

Only checks `key_spec == ECC_NIST_EDWARDS25519`. Does not verify `key_metadata.enabled()` or `key_usage() == SIGN_VERIFY`. A disabled key, or a key pending deletion, will pass the availability check.

Location

https://github.com/solana-foundation/solana-keychain/blob/b30253731d52cad8d71e853816183ba6188f5d61/rust/src/aws_kms/mod.rs#L195-L209

Relevant Code

```
/// mod.rs L195-L209
        match result {
            Ok(response) => {
                // Verify the key spec is ECC_NIST_EDWARDS25519
                if let Some(key_metadata) = response.key_metadata() {
                    if let Some(key_spec) = key_metadata.key_spec() {
                        // Check if key spec matches ECC_NIST_EDWARDS25519
                        // The SDK may represent this as a typed enum or as Unknown("ECC_NIST_EDWARDS25519")
                        let key_spec_str = key_spec.as_str();
                        return key_spec_str == AWS_KMS_KEY_SPEC;
                    }
                }
                false
            }
            Err(_) => false,
        }
```

Mitigation Suggestion

Also check `enabled` and `key_usage` fields.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-a33e7664d796b951ea9d26c5a2ade5e629d8c86d3d713a273e4b02d229fdf95cR233.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L10 Intermediate Key Material Not Zeroed (Memory Signer)

LOW FIXED

Description

All parsing paths allocate `Vec<u8>` buffers containing raw private key bytes that are never zeroed before being dropped. The `String` from `fs::read_to_string` also contains the raw JSON key representation. While the underlying `Keypair` may be zeroed via `zeroize`, all intermediate buffers persist in freed heap memory.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/memory/keypair_util.rs#L35-L36
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/memory/keypair_util.rs#L72
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/memory/keypair_util.rs#L96

Relevant Code

```
let decoded = bs58::decode(private_key)
    .into_vec()  // Vec<u8> with raw key bytes, never zeroed
```

Mitigation Suggestion

Use `zeroize::Zeroizing<Vec<u8>>` and `zeroize::Zeroizing<String>` for intermediate buffers.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-2177e12d887563015c232b0c6f4d753e8f99043bd5ec31f41b85659d50ee3e0aR34.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L11 Use-Before-Init Silently Uses Default Zero Public Key (Privy, Fireblocks)

LOW FIXED

Description

Both `PrivySigner` and `FireblocksSigner` require an `init()` call after construction to fetch the real public key from the remote API. Before `init()`, `pubkey()` returns `Pubkey::default()`. While the unified `Signer::from_privy()` and `Signer::from_fireblocks()` factory methods correctly call `init()`, direct construction via `PrivySigner::new()` or `FireblocksSigner::new()` leaves the signer in an uninitialized state with no runtime guard on signing operations.

For `sign_message()`, the call succeeds and returns a real signature, but the caller believes the signer's identity is the zero address. `FireblocksSigner::is_available()` does not check initialization state, so it can return `true` while `pubkey()` returns all zeros.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/privy/mod.rs#L48
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/fireblocks/mod.rs#L75

Relevant Code

```
/// privy/mod.rs L48
public_key: Pubkey::default(),

/// fireblocks/mod.rs L75
public_key: Pubkey::default(),
```

Mitigation Suggestion

1. Use `Option<Pubkey>` instead of `Pubkey::default()` and return `SignerError::ConfigError` from all signing operations when `None`. 2. Use the approach found in `AwsKmsSigner` to inline the `::init()` call inside the `::new` function.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-cbe50073e333b387b274cd03aac1049274f3588d76f1448d5b37c337ff2859c6R79

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L12 Arbitrary File Read via `from_private_key_string` (Memory Signer)

LOW FIXED

Description

The function attempts to interpret any input string as a file path first, before trying Base58 or U8Array parsing. There is no path validation, no canonicalization, and no restriction on what paths can be read. An attacker who can control the `private_key` parameter can: 1. Read arbitrary files (`/etc/passwd`, `/proc/self/environ`, symlinks) 2. Cause DoS by reading huge files (`/dev/zero` on Linux) 3. Use timing differences to probe file existence 4. Trigger unexpected behavior when a valid Base58 key string happens to match an existing filename

The impact is partially mitigated by most files not being in the correct private key format. However, it is dormant to the users that the API actually attempts to read files from disk. Ideally, if this behaviour is wanted, another method specific to file system reading should be considered

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/memory/keypair_util.rs#L17-L21

Relevant Code

```
pub fn from_private_key_string(private_key: &str) -> Result<Keypair, SignerError> {
    // Try to parse as a file path first
    if let Ok(file_content) = fs::read_to_string(private_key) {
        return Self::from_json_keypair(&file_content);
    }
    // ... try other formats
}
```

Mitigation Suggestion

Add a sister method that only does file reading:

```
pub fn from_private_key_file(...) -> ... { ... }
```

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-2177e12d887563015c232b0c6f4d753e8f99043bd5ec31f41b85659d50ee3e0aR28.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L13 `sign_transaction` and `sign_partial_transaction` are identical with no type-level distinction

LOW FIXED

Description

We found that all 10 backends implement `sign_partial_transaction` identically to `sign_transaction`, both of which delegate to the same internal `sign_and_serialize()`. Both return the same `SignedTransaction = (String, Signature)` type. `TransactionUtil::serialize_transaction` blindly calls `bincode::serialize` without checking whether all signature slots are filled or still contain `Signature::default()`.

A caller using `sign_transaction` in a multi-sig workflow reasonably assumes the returned transaction is fully signed and safe to submit. In reality, only one signature slot is filled while the remaining slots stay zeroed. The caller has no client-side way to distinguish between a complete and an incomplete transaction before submission.

Location

https://github.com/solana-foundation/solana-keychain/blob/d3ffc364c70429b9418568f72846a37999da7f4a/rust/src/traits.rs#L28-L60 https://github.com/solana-foundation/solana-keychain/blob/d3ffc364c70429b9418568f72846a37999da7f4a/rust/src/transaction_util.rs#L9-L14

Relevant Code

```
/// traits.rs L8
pub type SignedTransaction = (String, Signature);

/// traits.rs L28-L60
async fn sign_transaction(
    &self,
    tx: &mut Transaction,
) -> Result<SignedTransaction, SignerError>;

async fn sign_partial_transaction(
    &self,
    tx: &mut Transaction,
) -> Result<SignedTransaction, SignerError>;

/// transaction_util.rs L9-L14 -- no completeness check
pub fn serialize_transaction(transaction: &Transaction) -> Result<String, SignerError> {
    Ok(
        STANDARD.encode(bincode::serialize(transaction).map_err(|e| {
            SignerError::SerializationError(format!("Failed to serialize transaction: {e}"))
        })?),
    )
}
```

Mitigation Suggestion

Replace the two methods with a single method returning a discriminated type:

```
enum SignResult {
    Complete(SignedTransaction),
    Partial(PartialTransaction),
}
```

The implementation would inspect `transaction.signatures` for remaining `Signature::default()` slots and return the appropriate variant. This makes completeness a compile-time concern.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I1 Turnkey and Dfns Auth Use Node.js-Specific Crypto [TS]

INFO FIXED

Description

The Turnkey stamper uses `require('crypto').sign()` and the Dfns auth module uses `crypto.sign(undefined, ...)` from Node.js. These won't work in browsers or other runtime environments. CDP uses Web Crypto (`globalThis.crypto.subtle`) throughout, which is portable. This is a compatibility concern, not a security issue.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/turnkey/src/stamper.ts#L127
https://github.com/solana-foundation/solana-keychain/blob/c75ca49/typescript/packages/dfns/src/auth.ts#L81

Relevant Code

```
// stamper.ts L127
const signature = crypto.sign(undefined, Buffer.from(payload), privateKeyObj);

// auth.ts L81
const signature = crypto.sign(undefined, clientData, privateKeyPem);
```

Mitigation Suggestion

Use Web Crypto API for portability, or document the Node.js requirement.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/commit/8ee0909 and https://github.com/solana-foundation/solana-keychain/commit/fc44053.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I2 Fireblocks RSA PEM Re-parsed on Every API Call

INFO FIXED

Description

`EncodingKey::from_rsa_pem()` called per-request (up to 301 times per signing in worst case). Performance waste and unnecessary key material exposure in memory.

Location

https://github.com/solana-foundation/solana-keychain/blob/c75ca49/rust/src/fireblocks/jwt.rs#L50

Relevant Code

```
/// jwt.rs L50-L56
    let key = EncodingKey::from_rsa_pem(private_key_pem.as_bytes()).map_err(|_e| {
        #[cfg(feature = "unsafe-debug")]
        log::error!("Failed to parse RSA key: {_e}");

        SignerError::InvalidPrivateKey("Failed to parse RSA key".to_string())
    })?;
```

Mitigation Suggestion

Parse the PEM key once during construction and store the `EncodingKey` for reuse.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-4a67943159e45b3b9349e42ef2f8f2f5b1a249d05bcbdd18e48c0d7354ebc69aR66.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I3 No HTTP Request Timeouts Configured (All Remote Signers)

INFO FIXED

Description

All signers using `reqwest::Client::new()` or cloud SDK defaults have no timeout configured. A slow or unresponsive remote endpoint can block the calling async task indefinitely (or for the OS TCP timeout, typically minutes).

Location

All remote signer constructors using `reqwest::Client::new()`.

Relevant Code

```
client: reqwest::Client::new(),
```

Mitigation Suggestion

Accept an optional timeout configuration in each signer's config struct rather than hardcoding values. For example:

```
pub struct SignerClientConfig {
    /// HTTP request timeout (default: 30s)
    pub request_timeout: Option<Duration>,
    /// TCP connect timeout (default: 5s)
    pub connect_timeout: Option<Duration>,
}
```

Then build the client with the provided or default values:

```
let builder = reqwest::Client::builder()
    .timeout(config.request_timeout.unwrap_or(Duration::from_secs(30)))
    .connect_timeout(config.connect_timeout.unwrap_or(Duration::from_secs(5)));
```

This could be a shared struct reused across all signer configs, or individual fields on each config. Para already follows this pattern with `CLIENT_TIMEOUT` and `AVAILABILITY_TIMEOUT` constants.

Remediation

Fixed in https://github.com/solana-foundation/solana-keychain/pull/72/changes#diff-80381f991ebbf201e9eb999143d74cd3945e98f41cb690c67aca5d85c292cc93R79.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:
• Solana-specific vulnerabilities
• Access control issues
• Arithmetic errors and precision loss
• Race conditions and MEV opportunities
• Logic errors and edge cases
• Performance optimization opportunities
• Invariant violations
• Account confusion vulnerabilities
• Authority check omissions
• Token22 implementation risks and SPL-related pitfalls
• Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.